Réponses — TD2 Docker, partie 2

Firas Bouricha · Mastère Expert Cloud, Sécurité & Infrastructure · Ynov

les reponses sont trouvées en basant sur les documents fournis et le recherche sur internet et mes connaissances personnelles avec l'aide des LLM

Dépôt : https://github.com/firas-bouricha/docker-secure-deployment

Question 8.1

Expliquez pourquoi il est préférable de séparer le Dockerfile, les fichiers applicatifs, les données de la base et les configurations d'exécution.

> *Quels éléments seront incorporés à l'image et lesquels devront rester extérieurs à celle-ci ?*

Il est préférable de séparer le Dockerfile, les fichiers applicatifs, les données de la base et les configurations d'exécution pour plusieurs raisons :

- le code et le Dockerfile (app/) évoluent avec les versions de l'application et sont intégrés à l'image
- les scripts d'initialisation (db/init/) ne servent qu'au premier démarrage de MariaDB et sont montés, pas copiés dans l'image web
- les données de la base vivent dans un volume et doivent survivre à la reconstruction des conteneurs
- les paramètres d'exécution (compose.yaml) décrivent le déploiement et peuvent varier selon l'environnement (dev, prod) sans reconstruire l'image
- les secrets (secrets/) ne doivent jamais être versionnés ni intégrés à une image.

Séparer limite le contexte de construction au seul dossier app/, ce qui évite d'y envoyer les secrets ou les données, et permet de réutiliser la même image dans plusieurs environnements.

éléments seront incorporés à l'image :

- l'image de base PHP/Apache
- les extensions PHP (pdo_mysql)
- le code applicatif (public/).

À l'extérieur de l'image :

- les mots de passe (secrets montés dans /run/secrets/)
- les données MariaDB (volume db_data)
- le script SQL (monté dans le conteneur db)
- la configuration d'exécution (variables d'environnement, ports, réseaux dans compose.yaml).

Question 8.2 — Dockerfile

```
firas@Redmove01:~/td2-docker-app$ docker build -t ynov/td2-web:1.0 ./app
[+] Building 2.0s (9/9) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 147B
 => [internal] load metadata for docker.io/library/php:8.4-apache
 => [internal] load .dockerignore
 => => transferring context: 89B
 => [1/4] FROM docker.io/library/php:8.4-apache@sha256:be1326658a7d36dba6be79a71212d6ff9c5f854335b8ba77d714da1852c1b2ea
 => => resolve docker.io/library/php:8.4-apache@sha256:be1326658a7d36dba6be79a71212d6ff9c5f854335b8ba77d714da1852c1b2ea
 => [internal] load build context
 => => transferring context: 64B
 => CACHED [2/4] RUN docker-php-ext-install pdo_mysql
 => CACHED [3/4] WORKDIR /var/www/html
 => CACHED [4/4] COPY public/ /var/www/html/
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:8e9b9b2a3adbc00208f961250758cb0e3e8a6824fed5d09e7cc915a911ffc9ab
 => => exporting config sha256:ca5cf9ced9e5435891b1d865088f3a8ff9ad4468c17b9e18d301ba393f021873
 => => exporting attestation manifest sha256:d1cb31ae9c7b748cc1839bed285b9edf73385e352b834ceb2df88ce365d1c6b0
 => => exporting manifest list sha256:081cd74be51745c96cac90eeb2039936827d37547444ef99778e8de354b27d9a
 => => naming to docker.io/ynov/td2-web:1.0
 => => unpacking to docker.io/ynov/td2-web:1.0
```

- FROM php:8.4-apache : définit l'image de base (PHP 8.4 avec Apache, image officielle). Ses couches constituent la base sur laquelle les instructions suivantes ajoutent les leurs.
- WORKDIR /var/www/html : définit le répertoire de travail pour les instructions suivantes du Dockerfile et pour le conteneur au démarrage. Le dossier est créé s'il n'existe pas.
- COPY public/ /var/www/html/ : copie, au moment du build, le contenu de public/ (pris dans le contexte de construction) dans l'image, à l'emplacement servi par Apache. Cette instruction crée une nouvelle couche.
- ./app : c'est le contexte de construction, le répertoire envoyé au moteur Docker. Docker y cherche le Dockerfile par défaut, et les sources de COPY sont relatives à ce dossier. Les fichiers situés en dehors (comme secrets/) ne sont pas accessibles au build.
- Image ≠ conteneur : une image est un modèle immuable en lecture seule (couches de fichiers et configuration). Un conteneur est une instance en cours d'exécution de cette image, avec ses processus, son réseau et sa couche inscriptible. Plusieurs conteneurs peuvent être créés depuis la même image. Les modifications faites dans un conteneur ne changent pas l'image et disparaissent à sa suppression.

Question 8.3 — Structure d'une image

1- Identifiez les différentes étapes visibles dans l'historique.

> *2- Toutes les instructions Dockerfile produisent-elles nécessairement une couche contenant des modifications de fichiers ?*
> *3- Quelle différence existe-t-il entre les couches de fichiers et la configuration d'une image ?*

1- `docker image history ynov/td2-web:1.0` affiche les étapes de la plus récente à la plus ancienne :

- en haut, mes 3 instructions : `COPY public/ /var/www/html/` , `WORKDIR /var/www/html` et `RUN docker-php-ext-install pdo_mysql` ;
- en dessous, toutes les étapes héritées de l'image de base `php:8.4-apache` (installation de Debian, d'Apache, de PHP, puis `ENV` , `EXPOSE 80` , `CMD ["apache2-foreground"]` ...).

2- Non. Seules les instructions qui modifient des fichiers (`RUN` , `COPY` , `ADD`) créent une couche avec un contenu (taille supérieure à 0 B dans l'historique). Les instructions comme `ENV` , `EXPOSE` , `CMD` ou `WORKDIR` (le dossier existe déjà dans l'image de base) ne modifient que la configuration et apparaissent avec une taille de 0 B.

3- Les couches de fichiers contiennent le système de fichiers de l'image (programmes, bibliothèques, code PHP) : elles sont empilées et en lecture seule. La configuration est un ensemble de métadonnées (commande de démarrage, variables d'environnement, ports exposés, répertoire de travail, architecture `linux/amd64`) qui indique comment lancer le conteneur, sans contenir de fichiers.

Preuve — Historique des couches de l'image :

```
firas@Redmove01:~/td2-docker-app$ docker image history ynov/td2-web:1.0 --format 'table {{.CreatedBy}}\t{{.Size}}' | cut
 -c1-110 | head -n 14
CREATED BY                                      SIZE
COPY public/ /var/www/html/ # buildkit          20.5kB
WORKDIR /var/www/html                           4.1kB
RUN /bin/sh -c docker-php-ext-install pdo_my…   86kB
CMD ["apache2-foreground"]                      0B
EXPOSE map[80/tcp:{}]                           0B
WORKDIR /var/www/html                           4.1kB
COPY apache2-foreground /usr/local/bin/ # bu…   20.5kB
STOPSIGNAL SIGWINCH                             0B
ENTRYPOINT ["docker-php-entrypoint"]            0B
RUN /bin/sh -c docker-php-ext-enable sodium …   28.7kB
RUN /bin/sh -c docker-php-ext-enable opcache…   28.7kB
COPY docker-php-ext-* docker-php-entrypoint …   32.8kB
RUN /bin/sh -c set -eux;    savedAptMark="$(a…  59.2MB
firas@Redmove01:~/td2-docker-app$
```

Question 8.4

Docker consulte le fichier .dockerignore au moment de la construction de l'image, avant d'envoyer le contexte de construction au moteur Docker. Il utilise ce fichier pour déterminer quels fichiers et répertoires doivent être exclus du contexte de construction.

Si le contexte de construction contenait une archive de plusieurs gigaoctets, des fichiers temporaires ou des identifiants sensibles, cela pourrait entraîner plusieurs problèmes :

- L'envoi d'une archive volumineuse ralentirait considérablement le processus de construction et pourrait même provoquer des échecs si le système de fichiers ou le réseau ne peut pas gérer la taille.
- Les fichiers temporaires pourraient encombrer l'image finale, augmentant sa taille inutilement et rendant le déploiement plus lent.
- Les identifiants sensibles inclus dans le contexte de construction pourraient être accidentellement intégrés à l'image, exposant ainsi des informations confidentielles à toute personne ayant accès à l'image, ce qui constitue un risque de sécurité majeur.

.dockerignore est donc un outil utile pour exclure certains fichiers du contexte de construction, mais il ne remplace pas une politique de protection des secrets. Une politique de protection des secrets implique des pratiques de sécurité plus larges, telles que le stockage sécurisé des secrets, l'utilisation de gestionnaires de secrets, la rotation régulière des clés et l'accès restreint aux informations sensibles.

Question 8.5

> *1- Que se passe-t-il si vous redémarrez MariaDB avec un volume contenant déjà une base de données ?*
> *2- Le script SQL sera-t-il exécuté une seconde fois ?*
> *3- Comment appliqueriez-vous une évolution de schéma à une base déjà en production ?*

Réponses

1- Si vous redémarrez MariaDB avec un volume contenant déjà une base de données, le serveur MariaDB détectera que la base de données existe déjà et ne réinitialisera pas les données. Il utilisera les données existantes dans le volume pour continuer à fonctionner normalement.

2- Non, le script SQL ne sera pas exécuté une seconde fois. Les scripts d'initialisation sont conçus pour être exécutés uniquement lors de la première création de la base de données. Si la base de données existe déjà, MariaDB ignore ces scripts pour éviter de réinitialiser ou de corrompre les données existantes.

3-j'ai jamais fait ca personnelement mais selon mes recherche Le script 01-init.sql ne sert qu'au tout premier démarrage. Sur une base qui existe déjà, il n'est plus jamais exécuté, donc le modifier ne change rien.
Pour faire évoluer la base en production, il faut :

- Écrire un nouveau script qui contient seulement la modification (par exemple ajouter une colonne), et le numéroter (002-...sql) pour garder l'historique des changements.
- Faire une sauvegarde de la base avant de toucher à quoi que ce soit(toujours on sait jamais).
- Tester le script sur une copie de la base, pas directement en production.
- L'appliquer sur la vraie base, par exemple avec docker compose exec, ou avec un outil de migration (Flyway, Liquibase) qui retient les scripts déjà passés.
- Vérifier que l'application fonctionne toujours après la modification .

Question 8.6 — Gestion des secrets

> *1- Pourquoi inscrire un mot de passe avec l'instruction ENV d'un Dockerfile présente-t-il un risque ?*
> *2- Quelle différence existe-t-il entre une variable d'environnement, un fichier .env et un secret monté dans /run/secrets/ ?*

1- Inscrire un mot de passe avec l'instruction ENV d'un Dockerfile présente un risque car cette information est stockée dans l'image Docker. Toute personne ayant accès à l'image peut extraire le mot de passe en inspectant l'image ou en utilisant des commandes comme `docker history` ou `docker inspect`. Cela expose les informations sensibles à des utilisateurs non autorisés et constitue une faille de sécurité.

2- La différence entre une variable d'environnement, un fichier .env et un secret monté dans /run/secrets/ réside principalement dans la manière dont ils sont gérés et sécurisés :

- Une variable d'environnement est définie dans le conteneur et peut être visible par tous les processus du conteneur. Elle peut être définie dans le Dockerfile, dans le fichier docker-compose.yml ou via la ligne de commande lors du démarrage du conteneur. Cependant, elle n'est pas sécurisée et peut être exposée si le conteneur est compromis.

- Un fichier .env est un fichier texte qui contient des paires clé-valeur pour les variables d'environnement. Il est généralement utilisé pour stocker des configurations et peut être lu par Docker Compose pour définir les variables d'environnement. Cependant, si ce fichier est versionné ou partagé, il peut également exposer des informations sensibles.
- Un secret monté dans /run/secrets/ est une méthode plus sûre pour gérer les informations sensibles. Le secret est fourni au conteneur sous forme de fichier, et seulement aux conteneurs qui en ont besoin. Il n'est pas stocké dans l'image et n'apparaît pas dans les variables d'environnement ni dans `docker inspect`. Attention : avec Docker Compose, ce fichier reste en clair sur l'hôte (pas de chiffrement), voir Q8.20.

Question 8.7 — Diagnostic et reconstruction

> *1-Quel message permettait d'identifier l'origine de la panne ?*
> *2-Pourquoi fallait-il modifier l'image plutôt que simplement redémarrer le conteneur ?*
> *3-Quelle différence existe-t-il entre docker compose restart et docker compose up --build -d ?*

1-Le message qui révélait la panne : l'application répondait 503 {"status":"unavailable"}, et les journaux du service web (docker compose logs web) contenaient could not find driver. PHP ne trouvait pas le pilote MySQL. php -m l'a confirmé : seul pdo_sqlite était présent, pas pdo_mysql.

2- Pourquoi modifier l'image et non redémarrer ? Le pilote manquait dans l'image elle-même. Redémarrer le conteneur relance la même image, donc la même panne. Et l'installer à la main dans le conteneur serait perdu à sa suppression, en plus de ne pas être reproductible. Il fallait donc l'ajouter dans le Dockerfile et reconstruire l'image.

3- docker compose restart redémarre les conteneurs existants sans reconstruire l'image. Les modifications dans le Dockerfile ne sont pas prises en compte. docker compose up --build -d reconstruit l'image si nécessaire et recrée les conteneurs avec la nouvelle image, prenant en compte les changements du Dockerfile.

Preuve — pdo_mysql installé après correction du Dockerfile :

```
firas@Redmove01:~/td2-docker-app$ docker compose exec -T web php -m | grep -i pdo
PDO
pdo_mysql
pdo_sqlite
firas@Redmove01:~/td2-docker-app$
```

Preuve — Écriture (201) puis lecture des événements :

```
firas@Redmove01:~/td2-docker-app$ curl -s -i -X POST http://127.0.0.1:8080/ -H 'Content-Type: application/json' -d '{"me
ssage":"Preuve capture TD2"}'
HTTP/1.1 201 Created
Date: Thu, 29 Oct 2026 15:28:01 GMT
Server: Apache/2.4.69 (Debian)
X-Powered-By: PHP/8.4.27
Content-Length: 27
Content-Type: application/json; charset=utf-8

{"status":"created"}// v1.1
firas@Redmove01:~/td2-docker-app$ curl -s http://127.0.0.1:8080/
{
    "status": "ok",
    "events": [
        {
            "id": 1,
            "message": "Application initialis\u00e9e",
            "created_at": "2026-10-27 11:47:13"
        },
        {
            "id": 2,
            "message": "Premier d\u00e9ploiement Docker",
            "created_at": "2026-10-27 11:47:13"
        },
        {
            "id": 3,
            "message": "Infrastructure op\u00e9rationnelle",
            "created_at": "2026-10-27 11:47:13"
        },
        {
            "id": 4,
            "message": "Test de persistance Docker",
            "created_at": "2026-10-27 13:45:50"
        },
        {
            "id": 5,
            "message": "V\u00e9rification finale",
            "created_at": "2026-10-29 10:28:48"
        },
        {
            "id": 6,
            "message": "Test final",
            "created_at": "2026-10-29 13:46:45"
        },
        {
            "id": 7,
            "message": "Preuve capture TD2",
            "created_at": "2026-10-29 15:28:01"
        }
    ]
}
firas@Redmove01:~/td2-docker-app$
```

Question 8.8 — Résolution des services

> *1- Pourquoi le nom db permet-il à PHP de joindre MariaDB ?*
> *2- Est-il nécessaire de connaître son adresse IP ?*
> *3- Que se passe-t-il si le conteneur MariaDB est recréé avec une nouvelle adresse ?*

1- Le nom db permet à PHP de joindre MariaDB grâce au système de résolution de noms intégré à Docker Compose. Chaque service défini dans le fichier compose.yaml est automatiquement ajouté à un réseau interne, et Docker Compose configure un DNS interne qui permet aux conteneurs de se résoudre par leurs noms de service. Ainsi, le conteneur web peut utiliser le nom db pour se connecter au conteneur MariaDB sans avoir besoin de connaître son adresse IP.

2- Il n'est pas nécessaire de connaître l'adresse IP du conteneur MariaDB. Les conteneurs peuvent communiquer entre eux en utilisant les noms de service définis dans le fichier compose.yaml, ce qui rend la configuration plus flexible et moins sujette aux changements d'adresses IP.

3- Si le conteneur MariaDB est recréé avec une nouvelle adresse IP, le conteneur web pourra toujours se connecter à MariaDB en utilisant le nom de service db.

Preuve — Résolution DNS de db et ports publiés :

```
firas@Redmove01:~/td2-docker-app$ docker compose exec -T web php -r 'echo gethostbyname("db"), PHP_EOL;'
172.21.0.2
firas@Redmove01:~/td2-docker-app$ docker compose ps db web --format 'table {{.Service}}\t{{.Status}}\t{{.Ports}}'
SERVICE   STATUS                       PORTS
db        Up About a minute (healthy)   3306/tcp
web       Up 49 seconds                 127.0.0.1:8080->80/tcp
firas@Redmove01:~/td2-docker-app$
```

Question 8.9 — Cloisonnement

> *1- Pourquoi séparer les réseaux frontend et backend ?*
>
> *2- Quelle serait la conséquence d'une publication du port 3306 avec ports: "3306:3306" ?*
>
> *3- L'absence de port publié signifie-t-elle qu'un service est inaccessible depuis tous les autres conteneurs ?*

1- Séparer les réseaux frontend et backend permet de cloisonner les communications entre les services. Le réseau frontend est destiné aux services qui doivent être accessibles depuis l'extérieur (comme le service web), tandis que le réseau backend est réservé aux services internes (comme la base de données). Cela renforce la sécurité et réduisant la surface d'attaque.

2- La publication du port 3306 avec ports: "3306:3306" exposerait le service MariaDB à l'extérieur du réseau Docker. Cela augmenterait le risque de compromission de la sécurité

3- L'absence de port publié signifie que le service n'est pas accessible depuis l'extérieur du réseau Docker, mais il reste accessible depuis les autres conteneurs du même réseau.

Preuve — Réseaux frontend / backend : db uniquement sur backend :

```
firas@Redmove01:~/td2-docker-app$ docker network ls --filter name=td2-docker-app_ --format 'table {{.Name}}\t{{.Driver}}
\t{{.Internal}}'
NAME                        DRIVER    INTERNAL
td2-docker-app_backend      bridge    true
td2-docker-app_default      bridge    false
td2-docker-app_frontend     bridge    false
td2-docker-app_monitoring   bridge    true
firas@Redmove01:~/td2-docker-app$ docker network inspect td2-docker-app_backend --format 'backend (internal={{.Internal}
}) :{{range .Containers}} {{.Name}}={{.IPv4Address}}{{end}}'
backend (internal=true) : td2-docker-app-web-1=172.21.0.3/16 td2-docker-app-db-1=172.21.0.2/16
firas@Redmove01:~/td2-docker-app$ docker inspect $(docker compose ps -q db) --format 'db est sur : {{range $k, $v := .Ne
tworkSettings.Networks}}{{$k}} {{end}}'
db est sur : td2-docker-app_backend
firas@Redmove01:~/td2-docker-app$ docker inspect $(docker compose ps -q web) --format 'web est sur : {{range $k, $v := .
NetworkSettings.Networks}}{{$k}} {{end}}'
web est sur : td2-docker-app_backend td2-docker-app_frontend
firas@Redmove01:~/td2-docker-app$
```

Question 8.10 — Analyse réseau

1- Distinguez :

- Le réseau interne des conteneurs.
- Le port d'écoute du processus Apache.
- Le port publié sur l'hôte.
- Le mécanisme de résolution DNS interne.

2- Expliquez pourquoi l'adresse IP d'un conteneur ne doit pas constituer une dépendance permanente de la configuration applicative.

1-

- Réseau interne : chaque conteneur a une adresse IP privée sur le réseau Docker. Seuls les conteneurs du même réseau se voient.
- Port d'Apache : Apache écoute sur le port 80 à l'intérieur du conteneur. L'hôte ne le voit pas directement.
- Port publié : 127.0.0.1:8080:80 redirige ton port 8080 vers le port 80 du conteneur. C'est la porte d'entrée depuis ta machine.
- DNS interne : Docker traduit le nom db en adresse IP. Ça permet à web de trouver la base sans connaître son adresse.
- Pourquoi pas d'IP fixe ? L'adresse IP change quand le conteneur est recréé. Si elle est écrite en dur, l'application ne fonctionne plus. Le nom db reste toujours le même.

Question 8.11 — Démarrage

1- Quelle différence existe-t-il entre :

- Un conteneur démarré.
- Un processus serveur actif.

> - *Un service réellement prêt à répondre aux demandes ?*
>
> *2- Pourquoi un conteneur peut-il apparaître comme running alors que l'application est inutilisable ?*

1-

- Conteneur démarré : Docker a lancé le conteneur. Il affiche running, mais ça ne dit pas si l'application marche.
- Processus serveur actif : le programme tourne, mais il peut encore être en train de s'initialiser. Par exemple, MariaDB prépare ses données avant d'accepter les connexions.
- Service prêt : le service répond vraiment aux demandes. C'est le seul niveau qui compte pour l'utilisateur.

2- Pourquoi running mais inutilisable ? Docker vérifie seulement que le processus est vivant, pas qu'il fonctionne. L'application peut être en cours d'initialisation, ou renvoyer des erreurs.

Question 8.12 — Disponibilité

> *1- Quel avantage présente service_healthy par rapport à service_started ?*
> *2- Cette configuration garantit-elle que MariaDB restera disponible pendant toute la durée de vie de l'application ?*
> *3- Quelle autre responsabilité doit être assumée par le code applicatif en cas d'interruption temporaire de la base ?*

1- Avantage de service_healthy : il indique que le service est prêt à répondre aux demandes, pas seulement qu'il a démarré. Cela permet de s'assurer que l'application ne tente pas de se connecter à une base qui n'est pas encore prête.

2- Non, cette configuration ne garantit pas que MariaDB restera disponible pendant toute la durée de vie de l'application. Elle ne fait que vérifier l'état au démarrage. Des interruptions peuvent survenir pour diverses raisons (redémarrage du conteneur, panne du serveur, etc.).

3- Responsabilité du code applicatif : le code doit gérer les interruptions temporaires de la base de données, par exemple en implémentant des mécanismes de reconnexion, des files d'attente de requêtes, ou des stratégies de tolérance aux pannes pour assurer la continuité du service

Preuve — Healthcheck MariaDB : healthy :

```
firas@Redmove01:~/td2-docker-app$ docker inspect --format 'db health = {{.State.Health.Status}}' $(docker compose ps -q
db)
db health = healthy
firas@Redmove01:~/td2-docker-app$ docker compose ps db --format 'table {{.Service}}\t{{.Status}}'
SERVICE   STATUS
db        Up About a minute (healthy)
firas@Redmove01:~/td2-docker-app$
```

Question 8.13 — Diagnostic d'incident

> *1- Pourquoi le serveur web peut-il rester actif alors que l'application ne fonctionne plus ?*
> *2- Quels éléments permettent de distinguer une panne réseau, une indisponibilité de MariaDB et une erreur dans le code PHP ?*
> *3- Quelle amélioration proposeriez-vous pour assurer une meilleure tolérance aux interruptions temporaires ?*

1- Le serveur web peut rester actif même si l'application ne fonctionne plus parce que le processus Apache peut être en cours d'exécution, mais l'application PHP qu'il sert peut rencontrer des erreurs ou des problèmes de connexion à la base de données. Ainsi, le serveur web peut répondre aux requêtes HTTP, mais l'application elle-même peut échouer à traiter ces requêtes correctement.

2- Pour distinguer une panne réseau, une indisponibilité de MariaDB et une erreur dans le code PHP, on peut examiner les journaux du serveur web et de la base de données. Une panne réseau peut se manifester par des erreurs de connexion ou des délais d'attente. Une indisponibilité de MariaDB peut être identifiée par des messages d'erreur spécifiques liés à la base de données, tandis qu'une erreur dans le code PHP peut générer des messages d'erreur ou des exceptions dans les journaux du serveur web.

3- Pour assurer une meilleure tolérance aux interruptions temporaires, on pourrait mettre en place des mécanismes de reprise automatique dans l'application, tels que des tentatives de reconnexion avec des délais d'attente progressifs, l'utilisation de files d'attente pour gérer les requêtes en attente, et la mise en place de tests de santé pour surveiller l'état de la base de données et du serveur web. De plus, l'utilisation de solutions de mise en cache ou de réplication de la base de données pourrait également améliorer la résilience de l'application face aux interruptions temporaires.

Preuve — Panne simulée : 503, cause dans les logs, puis reprise :

```
firas@Redmove01:~/td2-docker-app$ docker compose stop db
 Container td2-docker-app-db-1 Stopping
 Container td2-docker-app-db-1 Stopped
firas@Redmove01:~/td2-docker-app$ sleep 3; curl -s -i http://127.0.0.1:8080/
HTTP/1.1 503 Service Unavailable
Date: Thu, 29 Oct 2026 15:28:39 GMT
Server: Apache/2.4.69 (Debian)
X-Powered-By: PHP/8.4.27
Content-Length: 24
Connection: close
Content-Type: application/json; charset=utf-8

{"status":"unavailable"}
firas@Redmove01:~/td2-docker-app$ docker compose logs web --tail 4 | cut -c1-150
web-1  | 172.22.0.3 - - [29/Oct/2026:15:28:20 +0000] "GET / HTTP/1.1" 200 1236 "-" "Telegraf/1.40.2 Go/1.27.1"
web-1  | 172.22.0.3 - - [29/Oct/2026:15:28:30 +0000] "GET / HTTP/1.1" 200 1236 "-" "Telegraf/1.40.2 Go/1.27.1"
web-1  | [Thu Oct 29 15:28:39.795062 2026] [php:notice] [pid 17:tid 17] [client 172.22.0.1:44058] SQLSTATE[HY000] [2002]
 php_network_getaddresses: get
web-1  | 172.22.0.1 - - [29/Oct/2026:15:28:39 +0000] "GET / HTTP/1.1" 503 241 "-" "curl/8.5.0"
firas@Redmove01:~/td2-docker-app$ docker compose start db
 Container td2-docker-app-db-1 Starting
 Container td2-docker-app-db-1 Started
firas@Redmove01:~/td2-docker-app$ for i in $(seq 1 60); do s=$(docker inspect --format '{{.State.Health.Status}}' $(dock
er compose ps -q db)); [ "$s" = healthy ] && break; sleep 2; done; echo db=$s
db=healthy
firas@Redmove01:~/td2-docker-app$ curl -s -o /dev/null -w 'HTTP %{http_code}\n' http://127.0.0.1:8080/
HTTP 200
firas@Redmove01:~/td2-docker-app$
```

Question 8.14 — Types de stockage

1-Quelle différence existe-t-il entre :

- La couche inscriptible d'un conteneur.
- Un volume Docker nommé.
- Un bind mount.

> *2- Pourquoi avons-nous choisi un volume nommé pour /var/lib/mysql ?*
> *3- Dans quel cas un bind mount serait-il préférable?*

1-

- Couche inscriptible : espace temporaire créé dans le conteneur. Les données disparaissent quand le conteneur est supprimé.
- Volume nommé : espace géré par Docker, en dehors du conteneur. Les données restent même si le conteneur est supprimé.
- Bind mount : dossier de ta machine monté dans le conteneur. Tu vois et modifies directement les fichiers.

2- Nous avons choisi un volume nommé pour /var/lib/mysql afin de garantir la persistance des données de la base de données MariaDB. En utilisant un volume nommé, les données restent intactes même si le

conteneur MariaDB est supprimé ou recréé, ce qui est essentiel pour maintenir l'intégrité et la continuité des informations stockées dans la base de données.

3- Un bind mount serait préférable dans les cas où l'on souhaite un accès direct aux fichiers depuis l'hôte, par exemple pour le développement ou le débogage. Cela permet de modifier les fichiers de l'application ou de la configuration en temps réel sans avoir à reconstruire l'image ou redémarrer le conteneur. Cependant, il faut être prudent avec les bind mounts en production, car ils peuvent introduire des risques de sécurité et de stabilité si les fichiers de l'hôte sont modifiés de manière inattendue.

Preuve — Volume nommé db_data :

```
firas@Redmove01:~/td2-docker-app$ docker volume ls --filter name=td2-docker-app
DRIVER    VOLUME NAME
local     td2-docker-app_db_data
local     td2-docker-app_grafana_data
local     td2-docker-app_influx_data
firas@Redmove01:~/td2-docker-app$ docker volume inspect td2-docker-app_db_data
[
    {
        "CreatedAt": "2026-10-27T12:47:02+01:00",
        "Driver": "local",
        "Labels": {
            "com.docker.compose.config-hash": "e883f1301f78d5dec8336a9e33ebb09b6166b405f2779ab7e076d1cebdc0106e",
            "com.docker.compose.project": "td2-docker-app",
            "com.docker.compose.version": "5.5.1",
            "com.docker.compose.volume": "db_data"
        },
        "Mountpoint": "/var/lib/docker/volumes/td2-docker-app_db_data/_data",
        "Name": "td2-docker-app_db_data",
        "Options": null,
        "Scope": "local"
    }
]
firas@Redmove01:~/td2-docker-app$
```

Question 8.15 — Conservation des données

> *1- Pourquoi les enregistrements sont-ils conservés alors que les conteneurs ont été supprimés ?*

2- Que se passerait-il avec :
docker compose down -v
N'exécutez pas cette dernière commande sur votre environnement de travail.

1- Les enregistrements sont conservés parce que les données de la base de données MariaDB sont stockées dans un volume Docker nommé (db_data). Les volumes Docker sont indépendants du cycle de vie des conteneurs, ce qui signifie que même si les conteneurs sont supprimés, les données dans le volume restent intactes. Ainsi, lorsque vous recréez les conteneurs, ils peuvent accéder aux mêmes données persistantes.

2- La commande `docker compose down -v` supprime les conteneurs, les réseaux et les volumes associés à la composition. Si vous exécutez cette commande, le volume nommé (db_data) sera également supprimé, ce qui entraînera la perte de toutes les données stockées dans la base de données MariaDB.

Preuve — Suppression / recréation des conteneurs : données conservées :

```
firas@Redmove01:~/td2-docker-app$ docker compose down db web
 Container td2-docker-app-web-1 Stopping
 Container td2-docker-app-web-1 Stopped
 Container td2-docker-app-web-1 Removing
 Container td2-docker-app-web-1 Removed
 Container td2-docker-app-db-1 Stopping
 Container td2-docker-app-db-1 Stopped
 Container td2-docker-app-db-1 Removing
 Container td2-docker-app-db-1 Removed
 Network td2-docker-app_backend Removing
 Network td2-docker-app_frontend Removing
 Network td2-docker-app_frontend Resource is still in use
 Network td2-docker-app_backend Removed
firas@Redmove01:~/td2-docker-app$ docker compose up -d --build db web 2>&1 | tail -n 8
 Container td2-docker-app-web-1 Creating
 Container td2-docker-app-web-1 Created
 Container td2-docker-app-db-1 Starting
 Container td2-docker-app-db-1 Started
 Container td2-docker-app-db-1 Waiting
 Container td2-docker-app-db-1 Healthy
 Container td2-docker-app-web-1 Starting
 Container td2-docker-app-web-1 Started
firas@Redmove01:~/td2-docker-app$ for i in $(seq 1 60); do s=$(docker inspect --format '{{.State.Health.Status}}' $(dock
er compose ps -q db)); [ "$s" = healthy ] && break; sleep 2; done; echo db=$s
db=healthy
firas@Redmove01:~/td2-docker-app$ docker compose ps db web --format 'table {{.Service}}\t{{.Status}}'
SERVICE   STATUS
db        Up 13 seconds (healthy)
web       Up 2 seconds
firas@Redmove01:~/td2-docker-app$ curl -s http://127.0.0.1:8080/ | grep -E 'Test de persistance|Preuve capture'
            "message": "Test de persistance Docker",
            "message": "Preuve capture TD2",
firas@Redmove01:~/td2-docker-app$
```

15.3 : Sauvegarder la base

Principe : un volume persistant conserve les données, mais ne les sauvegarde pas. La sauvegarde est un fichier SQL créé avec `mariadb-dump`.

1. Export

Le mot de passe est stocké sur la machine hôte, dans `secrets/db_app_password`. Ce fichier n'est pas versionné (`.gitignore`). Docker Compose le monte dans le conteneur `db`, et la commande `mariadb-dump` le lit depuis là. Le mot de passe n'apparaît donc ni dans la ligne de commande, ni dans le fichier de sauvegarde.

Remarque : en local, les mots de passe sont dans des fichiers ignorés par Git. En production, on utiliserait un gestionnaire de secrets (par exemple HashiCorp Vault, AWS Secrets Manager ou les secrets Docker Swarm), car un fichier `.env` exposerait les valeurs dans `docker inspect` et dans l'environnement des processus.

Commande : `./backup.sh`

2. Vérification

Le script contrôle trois points :

- le fichier n'est pas vide ;
- la dernière ligne est `-- Dump completed` ;
- le mot de passe n'apparaît pas dans le fichier.

Automatisation

une tâche cron lance `backup.sh` tous les jours à 2h. Les traces sont écrites dans `backup/backup.log`.

vous trouver le script complet dans le fichier `backup.sh` à la racine du projet. Il est commenté pour expliquer chaque étape.

Question 8.16 — Reprise

> *1- Pourquoi un volume persistant ne protège-t-il pas contre une suppression accidentelle du volume lui-même ?*
> *2- Quelle différence existe-t-il entre persistance, sauvegarde et restauration ?*

1- Un volume persistant ne protège pas contre une suppression accidentelle du volume lui-même parce que la persistance signifie simplement que les données sont conservées indépendamment du cycle de vie des conteneurs. Cependant, si le volume est supprimé (par exemple avec `docker volume rm` ou `docker compose down -v`), toutes les données qu'il contient seront perdues. La persistance ne fournit pas de mécanisme de sauvegarde ou de récupération en cas de suppression du volume.

2- La différence entre persistance, sauvegarde et restauration est la suivante :

- Persistance : les données sont conservées dans un volume Docker, ce qui permet de les maintenir même si les conteneurs sont supprimés ou recréés. Cependant, cela ne protège pas contre la suppression du volume lui-même.
- Sauvegarde : il s'agit de créer une copie des données (par exemple un fichier SQL) qui peut être stockée en dehors du volume persistant. La sauvegarde permet de récupérer les données en cas de perte ou de corruption du volume.
- Restauration : il s'agit de recharger les données à partir d'une sauvegarde précédente. La restauration permet de récupérer les données dans leur état précédent en cas de perte ou de corruption.

Question 8.17 — Identité et privilèges

> *1- Quelle différence existe-t-il entre l'utilisateur déclaré pour le conteneur et les utilisateurs des processus Apache ?*
> *2- Pourquoi un processus exécuté avec les privilèges root présente-t-il un risque ?*
> *3- Un utilisateur root à l'intérieur du conteneur est-il nécessairement root sur l'hôte ?*

1- La différence entre l'utilisateur déclaré pour le conteneur et les utilisateurs des processus Apache réside dans le fait que l'utilisateur déclaré pour le conteneur est celui qui exécute le conteneur lui-même, tandis que les utilisateurs des processus Apache sont ceux qui exécutent les processus du serveur web à l'intérieur du conteneur. L'utilisateur du conteneur peut être configuré pour avoir des privilèges limités, tandis que les processus Apache peuvent s'exécuter sous un utilisateur spécifique (comme www-data) avec des permissions restreintes pour des raisons de sécurité.

2- Un processus exécuté avec les privilèges root présente un risque car il a un accès complet au système de fichiers et aux ressources du conteneur. Si un attaquant parvient à exploiter une vulnérabilité dans le processus root, il pourrait potentiellement compromettre l'ensemble du conteneur et accéder à des informations sensibles ou exécuter des actions malveillantes. Il est donc recommandé de limiter les privilèges des processus pour réduire la surface d'attaque

3- Un utilisateur root à l'intérieur du conteneur n'est pas nécessairement root sur l'hôte. Les conteneurs utilisent un mécanisme de virtualisation léger qui isole les processus et les utilisateurs du conteneur de ceux de l'hôte. Ainsi, même si un processus s'exécute en tant que root à l'intérieur du conteneur, il n'a pas les mêmes privilèges que l'utilisateur root sur l'hôte. Cependant, il est important de noter que certaines

configurations ou vulnérabilités peuvent permettre à un processus root dans le conteneur d'obtenir des privilèges élevés sur l'hôte, ce qui constitue un risque de sécurité.

Preuve — Utilisateurs Apache et capabilities réduites :

```
firas@Redmove01:~/td2-docker-app$ docker compose exec -T web id
uid=0(root) gid=0(root) groups=0(root)
firas@Redmove01:~/td2-docker-app$ docker compose top web
SERVICE  #   UID       PID     PPID    C   STIME  TTY  TIME      CMD
web      1   root      827486  827458  2   16:27  ?    00:00:01  apache2 -DFOREGROUND
web      1   www-data  827588  827486  0   16:27  ?    00:00:00  apache2 -DFOREGROUND
web      1   www-data  827589  827486  0   16:27  ?    00:00:00  apache2 -DFOREGROUND
web      1   www-data  827590  827486  0   16:27  ?    00:00:00  apache2 -DFOREGROUND
web      1   www-data  827591  827486  0   16:27  ?    00:00:00  apache2 -DFOREGROUND
web      1   www-data  827592  827486  0   16:27  ?    00:00:00  apache2 -DFOREGROUND
firas@Redmove01:~/td2-docker-app$ docker compose exec -T web grep Cap /proc/1/status
CapInh:    0000000000000000
CapPrm:    00000000000004c0
CapEff:    00000000000004c0
CapBnd:    00000000000004c0
CapAmb:    0000000000000000
firas@Redmove01:~/td2-docker-app$ docker inspect $(docker compose ps -q web) --format 'CapDrop={{.HostConfig.CapDrop}}
CapAdd={{.HostConfig.CapAdd}}  SecurityOpt={{.HostConfig.SecurityOpt}}'
CapDrop=[ALL]  CapAdd=[CAP_NET_BIND_SERVICE CAP_SETGID CAP_SETUID]  SecurityOpt=[no-new-privileges:true]
firas@Redmove01:~/td2-docker-app$
```

Question 8.18 — principe de moindre privilège

> *1- Pourquoi cap_drop: ALL ne constitue-t-il pas toujours une solution directement applicable ?*
> *2- En quoi le choix de l'image de base influence-t-il la capacité à exécuter le service sans privilèges ?*

1-cap_drop: ALL n'est pas toujours applicable directement : certains services ont besoin de quelques droits pour démarrer. Apache démarre en root, ouvre le port 80 puis change d'utilisateur : sans SETUID/SETGID, il ne peut pas passer ses processus sur l'utilisateur www-data et s'arrête (erreur du type « setgid: unable to set group id… Operation not permitted »). Il faut donc tout retirer, tester, puis rajouter uniquement ce qui est nécessaire.

2-Influence de l'image de base : une image prévue pour tourner sans root (utilisateur non privilégié, port > 1024 comme 8080) n'a besoin d'aucune capability. php:8.4-apache démarre en root sur le port 80, ce qui oblige à garder 3 capabilities. Le choix de l'image détermine donc le niveau de privilèges minimal possible.

Question 8.19 — Intégrité et versionnement

> *1- Pourquoi un tag ne garantit-il pas à lui seul l'identité immuable d'une image ?*
> *2- Comment utiliser un digest pour identifier une version précise ?*
> *3- En quoi les architectures AMD64 et ARM64 compliquent-elles la comparaison des images ?*

1- Un tag ne garantit pas l'identité immuable d'une image car il peut être réutilisé pour pointer vers une nouvelle version de l'image. Par exemple, le tag "latest" peut être mis à jour pour référencer une nouvelle image, ce qui signifie que deux images avec le même tag peuvent avoir des contenus différents à différents moments. Ainsi, se fier uniquement aux tags peut entraîner des incohérences et des problèmes de compatibilité.

2- Pour utiliser un digest pour identifier une version précise, on peut se référer à le digest SHA256 de l'image. Le digest est un identifiant unique basé sur le contenu de l'image, et il ne change pas tant que le contenu de l'image reste le même. En utilisant le digest, on peut s'assurer que l'on utilise exactement la même image, indépendamment des tags qui peuvent être réutilisés ou modifiés. Par exemple, on peut spécifier l'image avec son digest dans le Dockerfile ou dans le fichier docker-compose.yml pour garantir l'utilisation d'une version spécifique et immuable de l'image.

3- Une image officielle comme mariadb:11.4 est multi-architecture : le même tag pointe vers un index qui contient une variante par architecture (AMD64, ARM64…). Chaque variante a son propre contenu binaire, donc son propre digest et son propre identifiant local. Deux postes qui téléchargent le « même » tag (un PC Intel en AMD64 et un Mac Apple Silicon en ARM64) obtiennent ainsi des images différentes, avec des ID différents. Pour comparer correctement, il faut soit comparer le digest de l'index (`docker buildx imagetools inspect mariadb:11.4`), soit préciser l'architecture. Un digest identique, en revanche, désigne toujours exactement le même contenu.

Preuve — Provenance, architecture et digests des images :

```
firas@Redmove01:~/td2-docker-app$ docker compose images db web
CONTAINER              REPOSITORY          TAG                 PLATFORM            IMAGE ID            SIZE
   CREATED
td2-docker-app-db-1    mariadb             11.4                linux/amd64         d1891ede5a55        451MB
   2 days ago
td2-docker-app-web-1   ynov/td2-web        1.0                 linux/amd64         058db5d5d893        722MB
   5 hours ago
firas@Redmove01:~/td2-docker-app$ docker image inspect mariadb:11.4 --format 'mariadb:11.4  arch={{.Os}}/{{.Architecture
}}  created={{.Created}}'
mariadb:11.4  arch=linux/amd64  created=2026-10-26T23:25:13.991402881Z
firas@Redmove01:~/td2-docker-app$ docker image inspect mariadb:11.4 --format '{{json .RepoDigests}}'
["mariadb@sha256:d1891ede5a559f1086f55053bb8cc1569b0298338c4d72d71725a7f74f6ba620"]
firas@Redmove01:~/td2-docker-app$ docker image inspect ynov/td2-web:1.0 --format 'ynov/td2-web:1.0  arch={{.Os}}/{{.Arch
itecture}}  digests={{json .RepoDigests}}'
ynov/td2-web:1.0  arch=linux/amd64  digests=["ynov/td2-web@sha256:058db5d5d8934c2736960c86b6457eb993a3138f1b363de1affa4d
d348e6d13e"]
firas@Redmove01:~/td2-docker-app$
```

Question 8.20 — Protection des informations sensibles

> *1- Quel intérêt présente le montage d'un secret sous forme de fichier ?*
>
> *2- Pourquoi une solution Compose locale ne doit-elle pas être assimilée automatiquement à un coffre-fort centralisé de gestion des secrets ?*
>
> *3- Quelles solutions envisageriez-vous pour un déploiement en production ?*

Réponses

1-
Intérêt du secret en fichier : le mot de passe n'est ni dans l'image, ni dans compose.yaml, ni dans les variables d'environnement (donc invisible dans docker inspect et env). Il n'est monté que dans les conteneurs qui en ont besoin (web ne voit pas le mot de passe root), et l'application le lit au moment voulu.

2-
Pas un coffre-fort : avec Compose, le secret n'est qu'un fichier en clair sur l'hôte, monté dans le conteneur. Il n'y a ni chiffrement, ni contrôle d'accès centralisé, ni rotation automatique, ni journal des accès. Quiconque a accès à l'hôte ou au groupe docker peut le lire.

3-
En production : Docker Swarm secrets ou Kubernetes Secrets (chiffrés au repos), ou mieux un coffre centralisé : HashiCorp Vault, AWS Secrets Manager, Azure Key Vault. Ils offrent chiffrement, droits d'accès, rotation et audit.

Preuve — Secrets montés, absents de l'image et des fichiers :

```
firas@Redmove01:~/td2-docker-app$ docker compose exec -T web ls -l /run/secrets/
total 4
-rw-r--r-- 1 1000 1000 37 Oct 27 11:09 db_app_password
firas@Redmove01:~/td2-docker-app$ docker compose exec -T db ls -l /run/secrets/
total 8
-rw-r--r-- 1 1000 1000 37 Oct 27 11:09 db_app_password
-rw-r--r-- 1 1000 1000 37 Oct 27 11:09 db_root_password
firas@Redmove01:~/td2-docker-app$ docker image inspect ynov/td2-web:1.0 --format '{{json .Config.Env}}'
["PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin","PHPIZE_DEPS=autoconf \t\tdpkg-dev \t\tfile \t\tg++
 \t\tgcc \t\tlibc-dev \t\tmake \t\tpkg-config \t\tre2c","PHP_INI_DIR=/usr/local/etc/php","APACHE_CONFDIR=/etc/apache2","
APACHE_ENVVARS=/etc/apache2/envvars","PHP_CFLAGS=-fstack-protector-strong -fpic -fpie -O2 -D_LARGEFILE_SOURCE -D_FILE_OF
FSET_BITS=64","PHP_CPPFLAGS=-fstack-protector-strong -fpic -fpie -O2 -D_LARGEFILE_SOURCE -D_FILE_OFFSET_BITS=64","PHP_LD
FLAGS=-Wl,-O1 -pie","GPG_KEYS=AFD8691FDAEDF03BDF6E460563F15A9B715376CA 9D7F99A0CB8F05C8A6958D6256A97AF7600A39A6 0616E93D
95AF471243E26761770426E17EBBB3DD","PHP_VERSION=8.4.27","PHP_URL=https://www.php.net/distributions/php-8.4.27.tar.xz","PH
P_ASC_URL=https://www.php.net/distributions/php-8.4.27.tar.xz.asc","PHP_SHA256=dcf93923b0e38e0d5252b25b796520b5a47761a08
b3488b6d0b5d1115764d3f0"]
firas@Redmove01:~/td2-docker-app$ grep -rlF -f secrets/db_app_password -f secrets/db_root_password --exclude-dir=secrets
 --exclude='*.tar' . || echo 'OK : aucun mot de passe dans les fichiers du projet'
OK : aucun mot de passe dans les fichiers du projet
firas@Redmove01:~/td2-docker-app$
```

Question 8.21 — Cache Docker

> *1- Pourquoi l'installation de l'extension PHP peut-elle être récupérée depuis le cache alors que la copie du code doit être réexécutée ?*
>
> *2- Dans quel cas serait-il pertinent d'utiliser --no-cache ?*
>
> *3- Pourquoi l'ordre des instructions influence-t-il les performances de construction ?*

Réponses

Pourquoi RUN vient du cache et pas COPY : Docker réutilise une étape si l'instruction et tout ce qui la précède n'ont pas changé. RUN docker-php-ext-install n'a pas changé, il est donc repris du cache. Pour COPY, Docker compare le contenu des fichiers copiés : index.php a changé, donc l'étape est refaite, ainsi que toutes celles qui suivent.

--no-cache : il force à tout refaire. C'est utile pour récupérer les mises à jour de sécurité (un RUN apt-get en cache garderait d'anciens paquets), pour corriger un cache corrompu, ou pour vérifier que le build fonctionne vraiment de zéro.

L'ordre compte : dès qu'une étape change, toutes les suivantes sont reconstruites. On met donc en haut ce qui change rarement (image de base, extensions) et en bas ce qui change souvent (le code). Ainsi, une modification du code ne relance pas l'installation des extensions, qui est lente.

Preuve — Seul COPY est réexécuté après modification du code :

```
firas@Redmove01:~/td2-docker-app$ echo "// modification" >> app/public/index.php      # modification du code PHP uniquem
ent
firas@Redmove01:~/td2-docker-app$ docker build --progress=plain -t ynov/td2-web:cache-demo ./app
#5 [1/4] FROM docker.io/library/php:8.4-apache@sha256:be1326658a7d36dba6be79a71212d6ff9c5f854335b8ba77d714da1852c1b2ea
#5 DONE 0.1s
#6 [2/4] RUN docker-php-ext-install pdo_mysql
#6 CACHED
#7 [3/4] WORKDIR /var/www/html
#7 CACHED
#8 [4/4] COPY public/ /var/www/html/
#8 DONE 0.1s
#9 DONE 0.4s
firas@Redmove01:~/td2-docker-app$
```

Question 8.22 — Versionnement

> *1- Cette commande a-t-elle produit un nouveau contenu d'image ?*
> *2- Comment distinguer la création d'un tag et la construction réelle d'une nouvelle version applicative ?*
> *3- Quelle politique de versionnement proposeriez-vous pour les images distribuées en production ?*

Nouveau contenu ? Non. docker image tag ajoute seulement un nouveau nom vers la même image : même ID, mêmes couches, aucune copie.

Tag ou nouvelle version : un nouveau tag garde le même ID. Une vraie nouvelle version (docker build après une modification du code) produit un ID différent. On le vérifie avec docker image inspect --format '{{.Id}}'.

Politique en production :

versionnement sémantique MAJEUR.MINEUR.CORRECTIF (1.2.3), un numéro par build, jamais réutilisé ;
ne jamais déployer latest en production ;
lier chaque image au commit Git (tag 1.2.3-abc1234 ou label) ;
déployer par digest pour garantir l'image exacte.

Preuve — Un tag partage l'ID de l'image ; une nouvelle version a un ID différent :

```
firas@Redmove01:~/td2-docker-app$ docker image tag ynov/td2-web:1.0 ynov/td2-web:1.0-alias
firas@Redmove01:~/td2-docker-app$ docker image inspect --format "{{.Id}}  {{.RepoTags}}" ynov/td2-web:1.0 ynov/td2-web:1
.0-alias ynov/td2-web:1.1
sha256:058db5d5d8934c2736960c86b6457eb99  [ynov/td2-web:1.0 ynov/td2-web:1.0-alias]
sha256:058db5d5d8934c2736960c86b6457eb99  [ynov/td2-web:1.0 ynov/td2-web:1.0-alias]
sha256:41c7380655213231ebd0602a1d8e050a1  [ynov/td2-web:1.1]
firas@Redmove01:~/td2-docker-app$
```

Question 8.23 — Distribution

> *1- Quelle différence existe-t-il entre publier une image dans un registre, transférer une archive d'image et copier un conteneur existant ?*
> *2- Quels avantages un registre présente-t-il pour une infrastructure comportant plusieurs hôtes ?*

Réponses

1- Registre (docker push/pull) : l'image est publiée sur un serveur central. Chaque hôte la télécharge par nom et tag ou digest, et seules les couches manquantes sont transférées.
Archive (save/load) : l'image complète tient dans un fichier .tar, copié à la main (clé USB, scp). C'est utile hors ligne, mais il n'y a ni versionnement ni partage des couches.

Copier un conteneur (docker commit ou export) : on capture un conteneur avec ses modifications manuelles. Ce n'est pas reproductible (aucun Dockerfile), on peut y embarquer des données ou des secrets, et export perd l'historique et la configuration.

2-Avantages d'un registre avec plusieurs hôtes : une source unique pour tous les serveurs, le téléchargement à la demande (pull), le versionnement par tags et digests, le transfert des seules couches manquantes, la gestion des droits d'accès, le scan de vulnérabilités et l'intégration CI/CD et orchestrateur (Kubernetes).

Preuve — Export / import de l'image :

```
firas@Redmove01:~/td2-docker-app$ ls -lh td2-web.tar
-rw------- 1 firas firas 172M Oct 29 11:06 td2-web.tar
firas@Redmove01:~/td2-docker-app$ docker image load -i td2-web.tar
Loaded image: ynov/td2-web:1.1
firas@Redmove01:~/td2-docker-app$
```

Question 8.24 — Exploitation

> *1- Quelles informations permettent de savoir qu'un conteneur fonctionne ?*
> *2- Ces informations suffisent-elles à garantir que le service métier est disponible ?*
> *3- Pourquoi faut-il distinguer métriques système, métriques applicatives et journaux ?*

Réponses

1-Ce qui montre qu'un conteneur fonctionne : l'état running et healthy (docker compose ps), la consommation CPU et mémoire et le nombre de processus (docker stats), et des journaux sans erreur

2-Non, ces informations ne suffisent pas à garantir que le service métier est disponible. Un conteneur peut être en état running et healthy, mais l'application qu'il héberge peut rencontrer des erreurs ou des problèmes de connexion à d'autres services, ce qui rend le service métier indisponible pour les utilisateurs finaux.

3-Il faut distinguer métriques système, métriques applicatives et journaux car ils fournissent des informations différentes sur le fonctionnement du conteneur et de l'application. Les métriques système (CPU, mémoire, I/O) permettent de surveiller les ressources utilisées par le conteneur et de détecter des problèmes de performance. Les métriques applicatives (temps de réponse, nombre de requêtes, erreurs) permettent de mesurer la performance et la disponibilité du service métier. Les journaux fournissent des informations détaillées sur les événements et les erreurs survenus dans le conteneur et l'application, ce qui est essentiel pour le diagnostic et la résolution des problèmes. En combinant ces trois types d'informations, on obtient une vue complète de l'état du conteneur et de l'application, ce qui permet de prendre des décisions éclairées pour assurer la disponibilité et la performance du service métier.

18.2. Extension : infrastructure de supervision

Architecture : 3 services supplémentaires dans Compose, sur un réseau dédié monitoring (interne, sauf Grafana) :

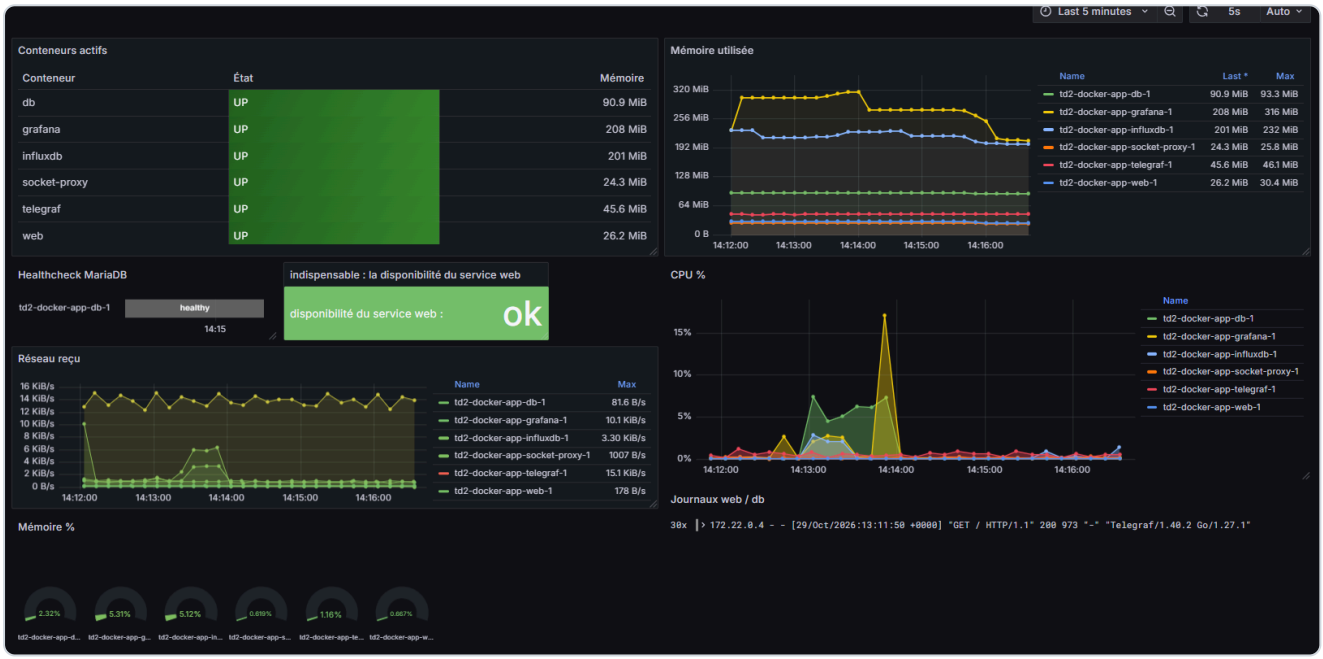
Telegraf : collecte les métriques des conteneurs via l'API Docker et les envoie à InfluxDB ;
InfluxDB : stocke les métriques dans un volume nommé (influx_data), sans port publié ;

Grafana : affiche les tableaux de bord, avec un volume grafana_data, publié seulement sur 127.0.0.1:3000, et un mot de passe admin passé en secret.

Risque du socket Docker : Telegraf a besoin de /var/run/docker.sock pour lire les statistiques. Or un accès à ce socket permet de piloter tout le démon Docker : créer un conteneur privilégié, monter / de l'hôte, donc devenir root sur l'hôte. Même monté en :ro, l'API reste utilisable. Pour limiter ce risque :

- passer par un proxy de socket (ex. docker-socket-proxy), qui n'autorise que la lecture des conteneurs et des statistiques
- ou collecter via cAdvisor et les cgroups
- ne pas exposer Telegraf ni InfluxDB sur le réseau, et ne pas les mettre sur backend.

Mise en œuvre et résultats



Lecture du dashboard :

Panneau	Rôle	Type de donnée
Conteneurs actifs	Conteneurs ayant envoyé une mesure dans les 30 dernières secondes	Système
Mémoire / CPU / Réseau	Consommation par conteneur	Système
Healthcheck MariaDB	Résultat du healthcheck Compose (§14.2)	Disponibilité
Disponibilité du service web	Code HTTP renvoyé par `http://web/` (sonde Telegraf)	Applicative
Journaux web / db	Logs Apache et MariaDB collectés par `inputs.docker_log`	Journaux

Telegraf collecte les métriques des conteneurs (CPU, mémoire, réseau), le code HTTP du service web et les journaux, InfluxDB les stocke et Grafana les affiche. Ces services tournent sur un réseau `monitoring` interne, et seul Grafana est publié, sur `127.0.0.1:3000`. Telegraf passe par un `docker-socket-proxy` en lecture seule, car un accès direct à `/var/run/docker.sock` donnerait le contrôle total de l'hôte. Lors du test `docker compose stop db`, `web` restait actif alors que la disponibilité passait à « Panne » : un

conteneur qui tourne ne prouve pas que le service fonctionne. Les journaux confirment la sonde Telegraf (`GET / 200`) toutes les 10 secondes.

Questions de synthèse

Q8.25 : Compose ou docker run

Compose décrit toute l'infrastructure (services, réseaux, volumes, secrets, dépendances) dans un seul fichier versionné, et la démarre avec une seule commande (docker compose up -d). Avec docker run, il faudrait retaper de longues commandes dans le bon ordre, ce qui est source d'erreurs et pas reproductible. Compose gère aussi automatiquement le réseau, le DNS entre services (db), l'ordre de démarrage (service_healthy) et la reconstruction (--build).

Q8.26 : 2 conteneurs, ni haute disponibilité ni reprise

Les 2 conteneurs ont des rôles différents : web et db ne sont pas des copies l'un de l'autre. Si db tombe, rien ne le remplace, et l'application est en panne (test de la section 14.3). Ils tournent sur un seul hôte : si la machine s'arrête, tout s'arrête. Sans politique restart ni orchestrateur, aucun conteneur n'est relancé automatiquement.

Q8.27 : LXC/LXD ou Docker

LXC/LXD fait des conteneurs système : un mini OS complet avec systemd, plusieurs services, SSH et apt, qu'on administre comme une petite VM, avec des données conservées dans le conteneur. Docker fait des conteneurs applicatifs : un seul processus par conteneur, une image construite par Dockerfile, et un conteneur jetable, recréé plutôt que réparé, avec les données dans des volumes. Les deux partagent le noyau de l'hôte et utilisent les mêmes mécanismes : namespaces et cgroups.

Q8.28 : plusieurs machines, tolérance aux pannes, mise à jour sans coupure

Limites actuelles : tout tourne sur un seul hôte (point unique de défaillance), avec une seule instance de chaque service. Le volume db_data est local à la machine. Une mise à jour recrée le conteneur et provoque une coupure, et rien ne relance automatiquement un service tombé.
Ce qu'un orchestrateur (Kubernetes, Docker Swarm) doit fournir :

- un cluster de plusieurs nœuds, avec placement et redéplacement automatique des conteneurs
- des réplicas et un répartiteur de charge
- le redémarrage automatique (auto-réparation) grâce aux sondes de santé
- les mises à jour progressives (rolling update) avec retour arrière
- un stockage partagé ou répliqué
- une gestion centralisée des secrets
- la mise à l'échelle automatique (autoscaling).